

Manual Técnico

Diagrama de Componentes UML:

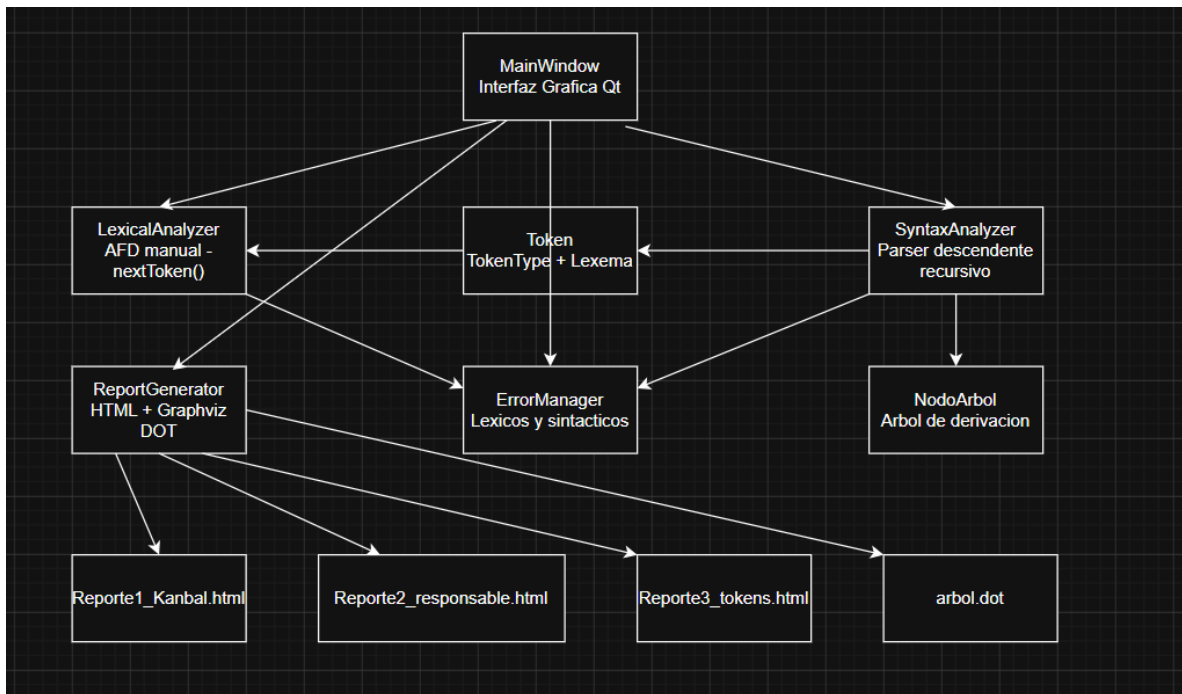
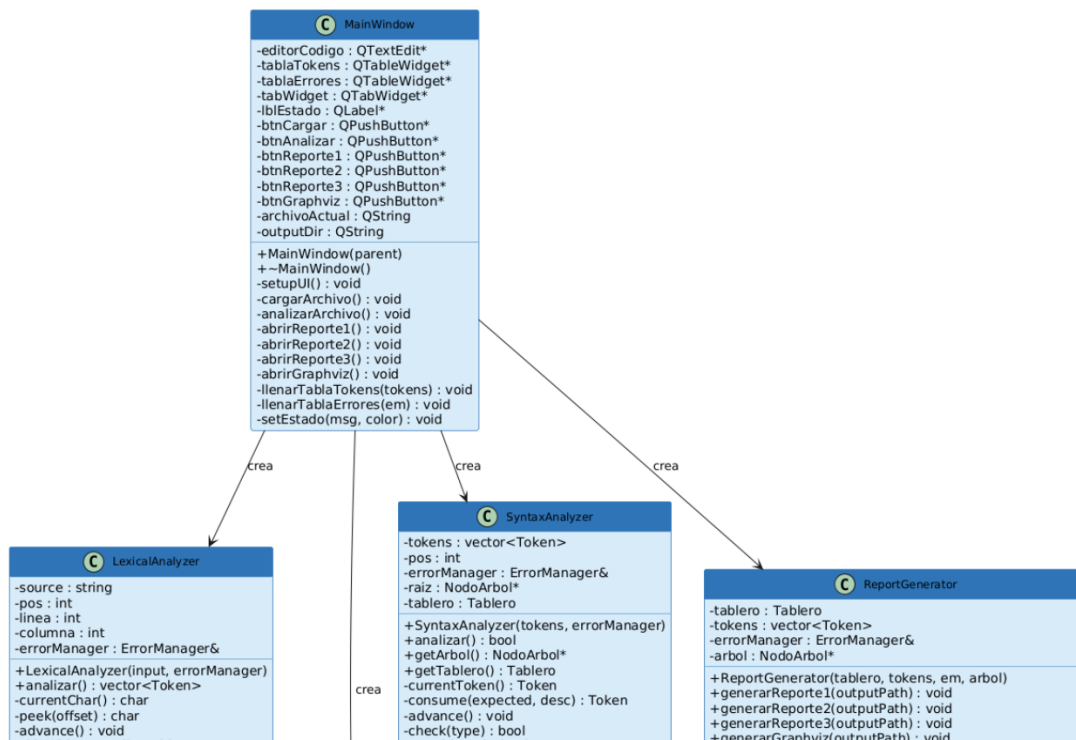


Diagrama de Clases:



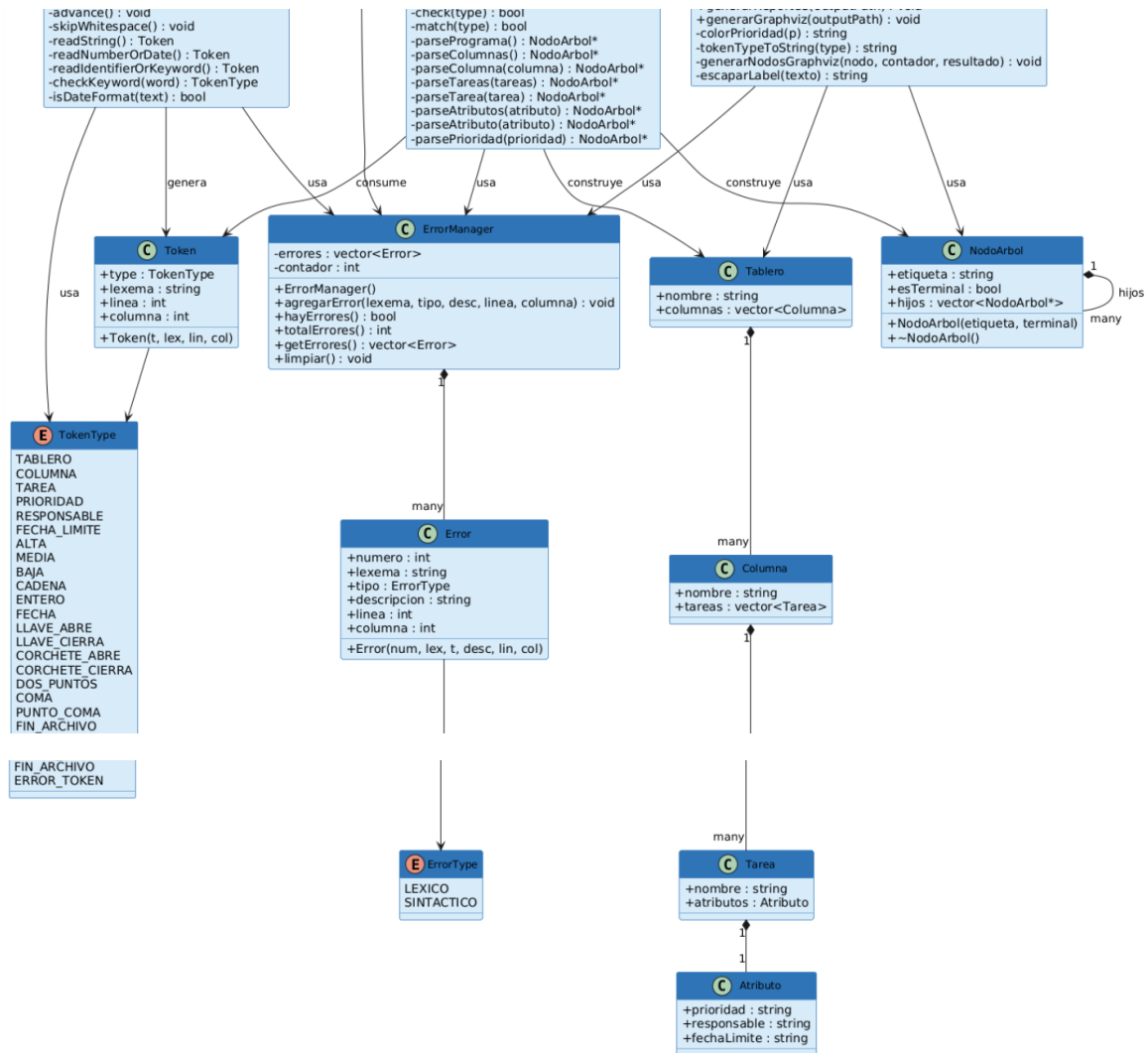
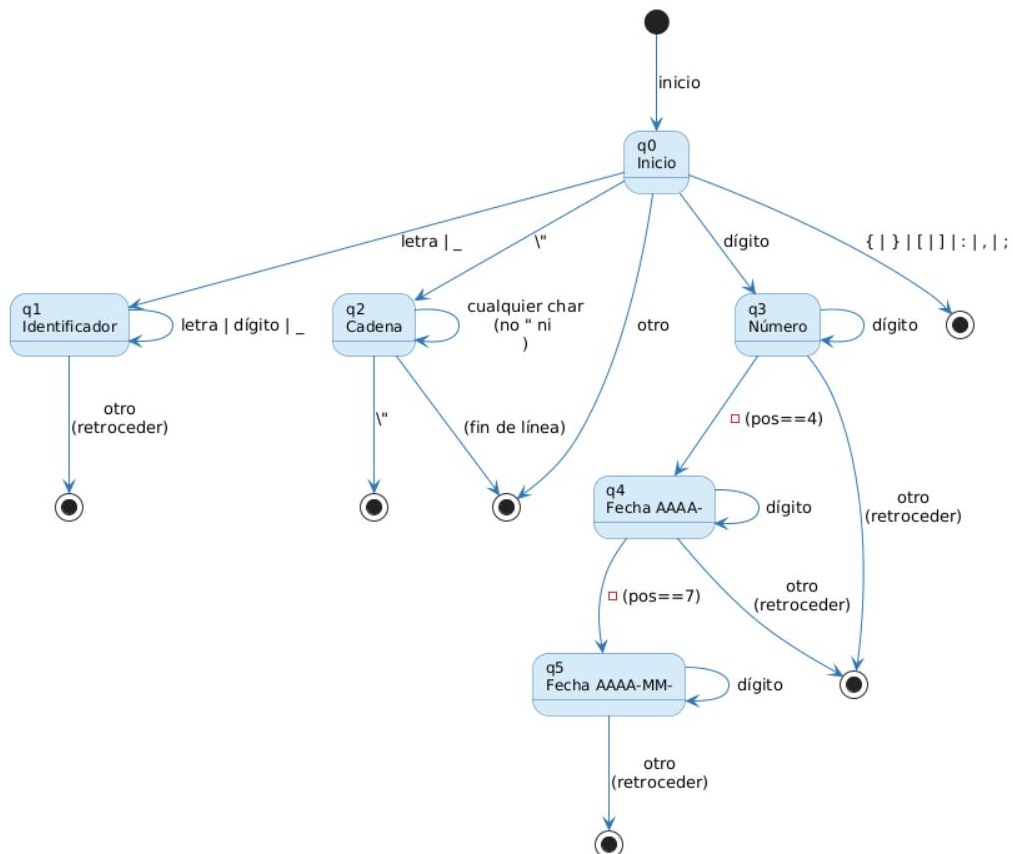


Diagrama del AFD con todos los estados y transiciones:



Gramática libre de contexto completa:

<programa> ::= TABLERO CADENA "{" <columnas> "}" ";"

<columnas> ::= <columna> <columnas>
| <columna>

<columna> ::= COLUMNA CADENA "{" <tareas> "}" ";"

<tareas> ::= <tarea> ";" <tareas>
| <tarea>

<tarea> ::= tarea ":" CADENA "[" <atributos> "]"

<atributos> ::= <atributo> ";" <atributos>
| <atributo>

<atributo> ::= prioridad ":" <prioridad>
| responsable ":" CADENA
| fecha_limite ":" FECHA

<prioridad> ::= ALTA
| MEDIA
| BAJA

Descripción del parser y estrategia de recuperación de errores:

Descripción del Parser

El parser implementado es un **analizador sintáctico descendente recursivo** (top-down recursive descent parser) desarrollado manualmente en C++ dentro de la clase SyntaxAnalyzer. Este tipo de parser funciona comenzando desde el símbolo inicial de la gramática (<programa>) y va expandiendo cada producción de forma recursiva hasta llegar a los tokens terminales.

Cada producción de la gramática libre de contexto tiene una función correspondiente en el código:

Producción Función en C++

<programa> parsePrograma()

<columnas> parseColumnas()

<columna> parseColumna()

<tareas> parseTareas()

Producción Función en C++

<tarea> parseTarea()

<atributos> parseAtributos()

<atributo> parseAtributo()

<prioridad> parsePrioridad()

El parser recibe como entrada el vector de tokens generado por el LexicalAnalyzer y construye simultáneamente el árbol de derivación (NodoArbol) y la estructura de datos del tablero (Tablero) que es utilizada posteriormente por el ReportGenerator para generar los reportes HTML.

El método principal consume() verifica que el token actual sea del tipo esperado. Si coincide, avanza al siguiente token. Si no coincide, registra un error sintáctico en el ErrorManager indicando qué token se esperaba y cuál se encontró, incluyendo la línea y columna exacta.

Estrategia de Recuperación de Errores

El sistema implementa una estrategia de **recuperación por modo pánico por sincronización de delimitadores**. Esta estrategia permite que el analizador continúe procesando el resto del archivo después de encontrar un error, en lugar de detenerse en el primer problema.

La estrategia funciona de la siguiente manera:

En parseColumnas(): Si el token actual no es COLUMNA ni }, el parser salta el token inesperado con advance() y continúa intentando reconocer la siguiente columna. Esto permite recuperarse de tokens basura entre columnas.

En parseTareas(): Si el token actual no es tarea ni }, el parser salta el token inesperado con advance() y continúa intentando reconocer la siguiente tarea. Esto permite recuperarse de errores dentro de una columna sin perder las tareas siguientes.

En consume(): Cuando se espera un token específico y se encuentra otro, se registra el error con su posición exacta pero el parser no se detiene — continúa desde el token actual e intenta seguir con el análisis.

Esta estrategia garantiza que:

- El análisis recorre el archivo completo en una sola pasada
- Todos los errores son acumulados y reportados al finalizar
- Un error en una tarea no impide analizar las tareas siguientes
- Un error en una columna no impide analizar las columnas siguientes

Justificación de decisiones de diseño:

Lenguaje C++17 con Qt 6 Se eligió C++17 por su soporte nativo para estructuras de datos de la STL como `std::vector` y `std::map`, que simplifican el manejo de colecciones de tokens, errores y nodos del árbol. Qt 6 fue elegido como framework de interfaz gráfica por su amplia documentación, compatibilidad multiplataforma y facilidad para construir tablas y paneles de forma programática sin depender del diseñador visual.

AFD implementado manualmente sin `std::regex` El autómata finito determinista fue implementado directamente en el método `nextToken()` mediante una función de transición de estados explícita con variables de posición, línea y columna. Esta decisión permite un control total sobre el seguimiento de la posición exacta de cada carácter, lo cual es fundamental para reportar errores léxicos con precisión. El uso de `std::regex` fue descartado por no permitir este nivel de control y por ser una abstracción que oculta el mecanismo del AFD.

Parser descendente recursivo manual Se implementó un parser descendente recursivo en lugar de usar herramientas generadoras como ANTLR o Bison. Esta decisión permite comprender y controlar completamente cada producción de la gramática, facilita la construcción simultánea del árbol de derivación y la estructura del tablero, y permite implementar estrategias de recuperación de errores personalizadas. Cada función del parser corresponde directamente a una producción de la gramática, haciendo el código legible y fácil de depurar.

Separación en clases independientes El sistema fue dividido en seis clases con responsabilidades bien definidas: `Token`, `LexicalAnalyzer`, `SyntaxAnalyzer`, `ErrorManager`, `ReportGenerator` y `MainWindow`. Esta separación sigue el principio de responsabilidad única, facilita las pruebas individuales de cada componente y permite modificar un módulo sin afectar a los demás.

ErrorManager centralizado Se diseñó una clase dedicada exclusivamente al manejo de errores en lugar de manejarlos directamente en el lexer o el parser. Esta decisión permite que ambos analizadores compartan el mismo gestor de errores, acumulen todos los errores en una sola pasada y los reporten de forma unificada al finalizar el análisis completo.

Generación de reportes HTML con CSS embebido Los tres reportes HTML se generan con CSS embebido en lugar de archivos externos. Esta decisión garantiza que los reportes sean completamente portables y puedan abrirse en cualquier navegador sin depender de rutas relativas o archivos adicionales.

Árbol de derivación con Graphviz Se eligió el formato DOT de Graphviz para representar el árbol de derivación porque permite generar visualizaciones jerárquicas complejas de forma automática sin necesidad de calcular coordenadas manualmente. Los nodos terminales y no terminales se diferencian por color para facilitar la lectura del árbol.

Reportes solo en análisis exitoso Se decidió generar los reportes del tablero Kanban, carga por responsable y árbol de derivación únicamente cuando el análisis no presenta errores. Esta decisión garantiza que los reportes siempre muestren información consistente y válida, evitando confusión al usuario con datos parciales o incorrectos.